CHESTNUT: A QoS Dataset for Mobile Edge Environments

Guobing Zou, Fei Zhao, Shengxiang Hu

***Abstract*—Quality of Service (QoS) is an important metric to measure the performance of network services. Nowadays, it is widely used in mobile edge environments to evaluate the quality of service when mobile devices request services from edge servers. QoS usually involves multiple dimensions, such as bandwidth, latency, jitter, and data packet loss rate. However, most existing QoS datasets, such as the common WS-Dream dataset, focus mainly on static QoS metrics of network services and ignore dynamic attributes such as time and geographic location. This means they should have detailed the mobile device's location at the time of the service request or the chronological order in which the request was made. However, these dynamic attributes are crucial for understanding and predicting the actual performance of network services, as QoS performance typically fluctuates with time and geographic location. To this end, we propose a novel dataset that accurately records temporal and geographic location information on quality of service during the collection process, aiming to provide more accurate and reliable data to support future QoS prediction in mobile edge environments.**

***Index Terms*—Quality of service, mobile edge environments, QoS datasets, time and geographic location**

IN the digital era, Quality of Service (QoS) [1] has emerged as a fundamental concern in the realm of network services. It is influenced by various factors, including service deployment conditions, user request locations, and the adaptability of the network environment [2]. As users' expectations for online services continue to rise—especially in real-time applications—QoS has a direct impact on user experience and satisfaction. Consequently, research on effective monitoring, prediction, and optimization of QoS has become a critical focus for both academia and industry. Key metrics of QoS, such as latency, bandwidth, packet loss rate, and availability, collectively provide a comprehensive evaluation of service performance [3], [4]. In the field of web services, numerous researchers have dedicated themselves to predicting and enhancing QoS through diverse algorithms to ensure stable and reliable services, particularly under high-load conditions [5]. As the number of users and request frequencies increases, traditional web service architectures often encounter challenges such as high latency and insufficient bandwidth, which adversely affect user experience [6]. Therefore, QoS is essential for web services, playing a vital role in improving user satisfaction and system efficiency [4]. Accurate QoS prediction can assist service providers in optimizing resource allocation [7], [8].

Collaborative filtering has become a prominent technique for predicting missing QoS values, typically categorized into memory-based and model-based approaches. Memory-based methods leverage historical QoS invocation data from user devices, evaluating similarities between users or services to form comparable neighborhoods for predicting missing QoS values [9]–[12]. However, these approaches often struggle with data sparsity, making it challenging to compute accurate similarities and thus impacting prediction quality. To mitigate this issue, model-based collaborative filtering aims to develop models from historical records, extracting latent semantic features of users and services to enhance prediction accuracy. Various studies have introduced innovative model-based approaches to tackle sparsity in QoS prediction for web services. For instance, Chen Wu et al. [10] proposed a time-aware and sparsity-tolerant method that combines historical QoS data with collaborative filtering, while Mingdong Tang et al. [13] employed location-based data smoothing to enhance prediction accuracy. Additionally, Kai Su et al. [14] developed a hybrid approach that integrates both memory-based and model-based techniques, utilizing neighbor information to address sparse data challenges. Qiong Zhang et al. [15] introduced a service ranking mechanism based on collaborative filtering, leveraging invocation and query histories to infer user behavior and calculate similarities. Temporal factors have also been integrated into various models to account for QoS fluctuations over time, with approaches such as Temporal Reinforced Collaborative Filtering (TRCF) [16] and dynamic graph neural collaborative learning showcasing superior performance in time-aware QoS prediction.

Despite these advancements, collaborative filtering-based approaches still face significant challenges related to data sparsity and the inability to effectively incorporate temporal and spatial contextual information [17], [18]. QoS performance is often influenced by multiple factors—such as time of day, user location, and network conditions—yet traditional collaborative filtering techniques fail to fully leverage this contextual information for more accurate predictions. This underscores the urgent need for exploring more advanced algorithms that can integrate these dynamic factors to enhance the effectiveness and accuracy of QoS prediction.

In recent years, there has been a notable increase in deep learning-based methods for QoS prediction in service recommendation. These approaches aim to improve prediction accuracy by effectively capturing the complex relationships among users and services. For example, Zhang et al. [19] introduced a two-stream deep learning model utilizing user and service graphs, while Jin et al. [20] developed a neighborhood-aware deep learning approach incorporating top-k neighbors. Additionally, Awanyo and Guermouche [21] proposed a deep

G. Zou, S. Lin, S. Hu, S. Duan are with the School of Computer Engineering and Science, Shanghai University, Shanghai 200444, China.
E-mail: gbzou@shu.edu.cn, shengxianghu@shu.edu.cn

neural network-based method for IoT service QoS prediction that combines Long Short-Term Memory (LSTM) networks [22] for service representation with ResNet for improved accuracy [23]. Zou et al. [24] proposed a two-tower deep neural network with residual connections to extract similar features of users and services, enhancing adaptive QoS prediction by merging deep learning with collaborative filtering.

Recent research has also shifted towards developing distributed QoS prediction models that prioritize data privacy and security while maintaining accuracy. The DISTINQT framework [25] encodes raw input data into non-linear latent representations, achieving performance comparable to centralized models. The FHC-DQP framework [26] integrates federated learning with hierarchical clustering to enhance prediction accuracy while preserving user privacy. Zou et al. proposed a novel model called Federated Residual Ladder Network (FRLN) [27] aimed at QoS prediction with a focus on data protection, combining the strengths of federated learning and residual networks to share learning results across multiple nodes while safeguarding user data privacy.

Given this context, it is crucial to efficiently and accurately predict QoS. Currently, researchers primarily rely on the WS-DREAM dataset [28] for model training and validation; however, this dataset is not well-suited for mobile edge environments. The QoS of edge devices may change dynamically as users move, contrasting with static predictions based on traditional web services, which makes it challenging for conventional methods to handle the complexities inherent in edge computing.

With the rapid advancement of 5G and cloud-edge environments, the rise of Everything-as-a-Service (XaaS) [29] has facilitated a gradual transition of services from the cloud to the edge. This shift significantly enhances service flexibility and responsiveness, enabling edge devices to communicate and compute services directly with edge systems, thereby avoiding the latency associated with routing through backhaul links to remote cloud servers. As a result, network latency is drastically reduced, providing a smoother user experience but also imposing greater demands on QoS prediction.

In this dynamic landscape, QoS prediction in mobile edge environments becomes particularly critical. The variability of edge devices and the diversity of user behaviors challenge traditional QoS prediction models to adapt to new requirements. Thus, there is an urgent need for a specialized QoS prediction dataset tailored for mobile edge environments to support more accurate model training and validation. This initiative will not only improve prediction accuracy but also lay a solid foundation for future research and further promote the development and application of edge computing technologies.

Therefore, we have constructed a QoS dataset specifically for mobile edge environments, addressing the need for QoS prediction in dynamic settings. This dataset incorporates user mobility, edge server resource load, and service diversity, providing a more realistic simulation of the operational environment. Additionally, we define a paradigm for calculating QoS values based on the unique characteristics of edge environments, thus establishing a robust data foundation for subsequent QoS prediction models in mobile edge scenarios. By utilizing this dataset, researchers can more effectively develop and validate models to tackle the intricate challenges posed by mobile edge computing. To facilitate reproducible research, the public release of CHESTNUT is available on Kaggle at https://www.kaggle.com/datasets/sorriso07/chestnut.

I. ORIGINAL DATASET DESCRIPTION

To create a high-quality dataset for predicting Quality of Service (QoS) in mobile edge environments, this study utilized two real-world datasets from Shanghai. One dataset is from the Shanghai Johnson Taxi, containing information such as the longitude, latitude, moving direction, and speed of the taxis on a specific day, which was used to simulate user mobile datasets. The other dataset is from Shanghai Telecom [30]–[32], providing the longitude and latitude of the base stations. Data from June 2014 was used to simulate the generation of edge server datasets.

A. Shanghai Johnson Taxi Dataset

The Shanghai johnson taxi dataset is an important resource for traffic research, containing real-time GPS and business status information from taxis in Shanghai. Each record in the dataset includes various fields: the vehicle ID, control word (where A indicates normal and M indicates alarm), business status (0 for normal and 1 for alarm), passenger status (0 for occupied and 1 for unoccupied), top light status (with values ranging from 0 for operation to 5 for out of service), road type (0 for ground road and 1 for express road), brake status (0 for no braking and 1 for braking), meaningless fields, reception date, GPS timestamp, longitude, latitude, speed, direction, the number of satellites, and additional meaningless fields. This dataset enables the analysis of urban traffic flow, travel patterns, and traffic management strategies. In this paper, we use only the gps time, latitude, longitude, speed, and direction of the cab to generate motion information about the mobile user.

B. Shanghai Telecom Dataset

This study utilized a telecommunications dataset provided by Shanghai Telecom, comprising over 7.2 million records of 9,481 mobile phones accessing the Internet through 3,233 base stations over six months.[1] The dataset includes six parameters: month, date, start time, end time, base station location (latitude and longitude), and user ID used within Shanghai Telecom.

This dataset can be used to evaluate solutions in mobile edge computing, such as edge server deployment, service migration, and service recommendation. In our research, we need to simulate the information of edge servers on base stations based on this dataset. Furthermore, considering the substantial volume of data, we only counted the data for one month and only focused on the geographic location information of Shanghai base stations.

[1] http://sguangwang.com/TelecomDataset.html

II. Data Preparation

CHESTNUT is built by integrating the two real-world Shanghai datasets above and then synthesizing the missing edge-side attributes required for mobile QoS research. Taxi trajectories are used to emulate mobile users, while telecom base stations are treated as candidate edge servers with fixed locations. Because the raw datasets were collected for traffic analysis and network operation rather than QoS modeling, several preprocessing steps are required before they can be used to generate user traces, server states, service types, and invocation records. The main configuration is summarized in Table I.

TABLE I
Simulation Parameters

Parameter	Value
Minimal latitude ϕ_-	31.050
Maximal latitude ϕ_+	31.372
Minimal longitude λ_-	121.259
Maximal longitude λ_+	121.640
Minimal server coverage radius r_-	400
Maximal server coverage radius r_+	3200
Number of users N_u	1000
Number of services N_s	1000
Maximal server resource level P_e	6
Maximal service demand level P_s	4
Time alignment interval Δt	30
Maximum system time T_{max}	21600
Minimum valid timestamps C_{min}	30
Maximum stationary interval S	3
Base delay Θ_{RT}	1.6
Base jitter Θ_{NJ}	160
History window size k	5

A. Edge Server Data

Edge server locations are extracted from the Shanghai Telecom dataset [30]–[32]. We keep only the base stations whose coordinates fall inside the target urban area bounded by $[\phi_-, \phi_+]$ and $[\lambda_-, \lambda_+]$. This produces a geographically dense deployment, which is important because mobile users should be able to move across overlapping coverage regions rather than frequently falling into uncovered zones. The resulting server map is illustrated in Figure 1.

The telecom records provide base station coordinates but not radio coverage radii. To approximate heterogeneous 4G and 5G edge access ranges, each selected site is assigned a radius sampled from $[r_-, r_+]$. Since geographic coverage is measured on the Earth surface rather than on a plane, we compute distances with the Haversine formula:

$$\mathrm{hav}(\theta) = \mathrm{hav}(\Delta\phi) + \cos(\phi_1)\cos(\phi_2)\,\mathrm{hav}(\Delta\lambda) \quad (1)$$

$$D = 2R \arcsin\left(\sqrt{\mathrm{hav}(\theta)}\right) \quad (2)$$

where R is the Earth radius, $\Delta\phi = \phi_2 - \phi_1$, and $\Delta\lambda = \lambda_2 - \lambda_1$.

Each server is further assigned three resource supply levels for computation, storage, and bandwidth. These values are integers in $[1, P_e]$, where larger values indicate stronger provisioning capacity. Instead of fixing device-specific hardware specifications, CHESTNUT models relative resource strength,

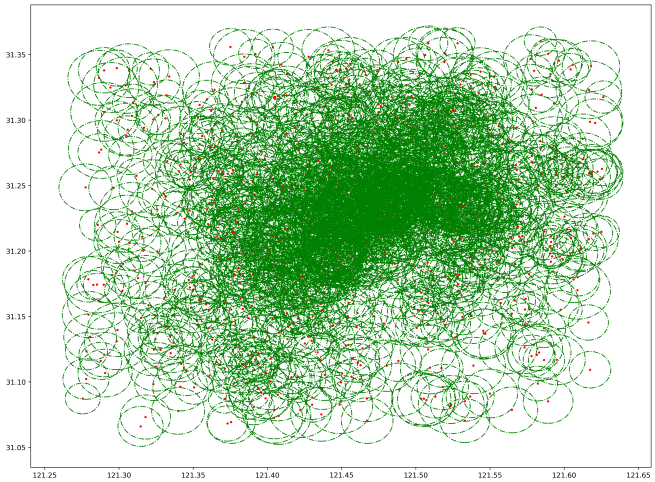

Fig. 1. Edge server distribution in the selected Shanghai area.

which is sufficient for downstream QoS prediction and makes the dataset easier to generalize. A server e_i is therefore represented as

$$e_i = \langle i, \lambda_i, \phi_i, r_i, \psi_{i,c}, \psi_{i,s}, \psi_{i,b} \rangle \quad (3)$$

B. Service Data

To model heterogeneous edge applications without binding the dataset to a particular software stack, CHESTNUT synthesizes N_s service types. Each service is described by three demand levels: computation, storage, and bandwidth. The three values are sampled from $[1, P_s]$, where a larger value means the service consumes more of the corresponding resource. Although at most P_s^3 unique demand combinations exist, services with the same demand tuple are still retained as different service IDs to mimic latent application differences that are not explicitly represented in the dataset.

This design keeps the service schema compact while preserving enough heterogeneity for QoS analysis. A sample of the service table is shown in Table II.

TABLE II
Sample Service Records

sid	computing	storage	bandwidth
0	1	4	2
1	4	2	3
2	3	1	4
3	2	3	1
4	4	4	2

C. User Data

The Shanghai Johnson Taxi dataset is used to emulate mobile users. After removing irrelevant columns, each retained record contains the taxi ID, GPS timestamp, longitude, latitude, instantaneous speed, and moving direction. These fields are enough to reconstruct fine-grained user mobility, but they still need to be aligned and filtered because taxis become active at different times and report GPS records at irregular intervals.

1) Temporal Alignment: The first issue is that the original trajectories are not synchronized. In CHESTNUT, the first GPS record of each taxi is mapped to the initial system timestamp, and the trajectory is then projected onto a discrete edge-system clock. Figure 2 shows the irregular reporting pattern in the raw data.

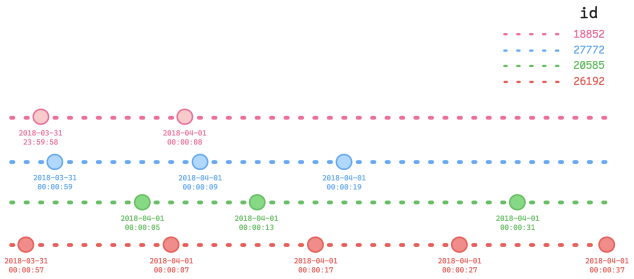

Fig. 2. Examples of irregular taxi GPS sequences before alignment.

To reduce temporal inconsistency, we further partition each trajectory into fixed windows of length Δt seconds. If several records fall into the same window, only the last one is kept as the valid snapshot for that timestamp. If no record appears in a window, the user is treated as temporarily absent from the monitored edge environment. The aligned result is illustrated in Figure 3.

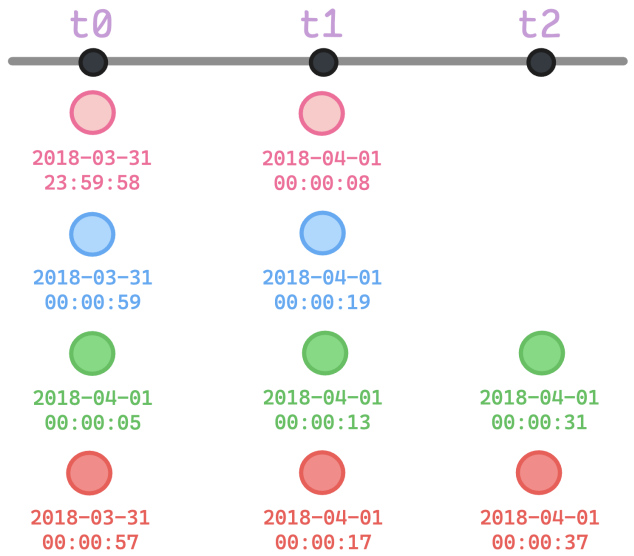

Fig. 3. Aligned user trajectories after fixed-window snapshotting.

For a user u, suppose the aligned trajectory contains τ_u valid timestamps. We denote its timestamp set by $K_u = \{\kappa_1, \kappa_2, \ldots, \kappa_{\tau_u}\}$, where

$$\kappa_i \in \left[0, \left\lfloor \frac{T_{max}}{\Delta t} \right\rfloor\right]. \tag{4}$$

This procedure yields user traces that are temporally comparable across the whole system.

2) Active User Selection: Even after alignment, many taxis contribute little useful motion diversity because they stay nearly static for long periods or appear only briefly. Such traces are weak supports for mobile QoS prediction, so we keep only the most informative users. First, we remove users whose longest stationary streak exceeds S timestamps or whose total number of valid timestamps is below C_{min}. Then, for each remaining user u, we compute four mobility-related indicators:

- τ_u: the number of valid timestamps.
- D_u: the great-circle distance between the first and last observed locations.
- ω_u^t: the number of servers covering user u at timestamp t.
- ν_u^t: the number of timestamps at which user u is covered by at least one server.

Users are ranked in descending lexicographic order by

$$\left(\tau_u, D_u, \sum_{t \in T_u} \omega_u^t, \sum_{t \in T_u} \nu_u^t\right), \tag{5}$$

which favors long-lived, highly mobile users that frequently move through densely covered areas. We then keep the top N_u users and reindex them. Figure 4 shows the resulting user distribution, while Figure 5 gives a local example of how a moving user traverses multiple overlapping server regions.

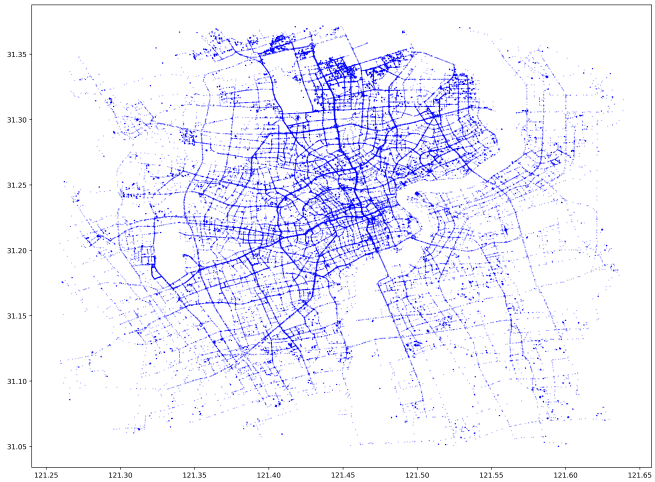

Fig. 4. Spatial distribution of the selected mobile users.

III. Synthetic Data Generation

After obtaining aligned user traces, server attributes, and service types, we synthesize dynamic service invocations and their QoS values. The generation process models not only where a user is located, but also how often it requests services, which server receives the requests, how server loads evolve, and how latency and jitter emerge from user-server interactions. Table IV lists the main symbols used below.

A. Invocation Sequence Synthesis

In real mobile edge systems, different users often exhibit correlated but non-identical request rhythms. To reflect this, we divide users into several request-pattern groups and assign each group a shared activity timeline. For a group g_i, all users in that group inherit the same active timestamp set T_{g_i}, which

TABLE III
SAMPLE USER RECORDS

id	timestamp	lon	lat	speed	direction
0	0	121.431827	31.306520	0.0	84
1	0	121.478397	31.321983	0.0	305
1	1	121.478383	31.321997	0.0	305
91	3	121.469835	31.285843	0.0	0
77	7	121.334412	31.264438	20.0	180

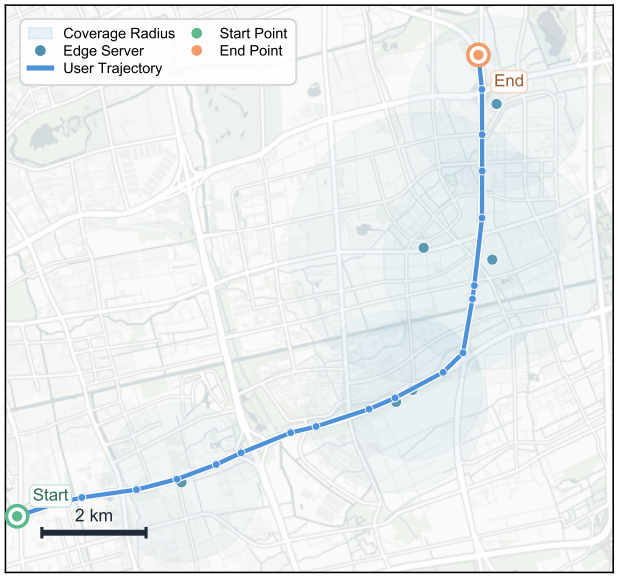

Fig. 5. A local example of user movement and overlapping edge coverage.

TABLE IV
MAIN NOTATIONS

Notation	Description
$\varphi_{s,c}, \varphi_{s,s}, \varphi_{s,b}$	service demand levels for computation, storage, and bandwidth
$\psi_{e,c}, \psi_{e,s}, \psi_{e,b}$	server supply levels for computation, storage, and bandwidth
R_e	coverage radius of server e
$\rho^t_{e,c}, \rho^t_{e,s}, \rho^t_{e,b}$	resource load rates of server e at timestamp t
q^t_u	number of requests issued by user u at timestamp t
v^t_u	instantaneous speed of user u at timestamp t
θ^t_u	moving direction of user u at timestamp t

captures coarse-grained common behaviors such as commuting peaks or periodic bursts.

Within the shared timeline, each user receives an individualized request-count signal generated by a Gaussian process [33] with additive white noise. For user u,

$$f_u(t) \sim \mathcal{GP}(0, k(t,t')) \tag{6}$$
$$\epsilon_u(t) \sim \mathcal{N}(0, \sigma_u^2), \qquad \sigma_u \sim \mathcal{U}(0.1, 0.3) \tag{7}$$
$$s_u(t) = f_u(t) + \epsilon_u(t) \tag{8}$$

where the kernel $k(t,t')$ is selected from a generator pool containing smooth, periodic, and bursty temporal patterns. The continuous signal is then shifted and scaled into a non-negative integer request count:

$$q^t_u = \left\lceil 20 \cdot \left(s_u(t) - \min_{\tau \in T_u} s_u(\tau) \right) \right\rceil \tag{9}$$

Each of the q^t_u requests is sent to the nearest server that covers user u at timestamp t. The target service is sampled from the service subset associated with the user's request-pattern group. In this way, users in the same group share similar global trends while still exhibiting user-specific local variations. An invocation record is defined as

$$\mathcal{R}_i = \langle u, e, s, t \rangle \tag{10}$$

and all records together form the invocation set $\mathcal{R}$.

B. Edge Server Load

Server loads should evolve continuously rather than being regenerated independently at each timestamp. For this reason, CHESTNUT uses a recursive load model. At the beginning of the system, each server is assigned an initial background load vector $\rho^0_e = [\rho^0_{e,c}, \rho^0_{e,s}, \rho^0_{e,b}]$. At timestamp $t-1$, the remaining supply levels of server e are

$$p^{t-1}_{e,c} = (1 - \rho^{t-1}_{e,c}) \cdot \psi_{e,c} \tag{11}$$
$$p^{t-1}_{e,s} = (1 - \rho^{t-1}_{e,s}) \cdot \psi_{e,s} \tag{12}$$
$$p^{t-1}_{e,b} = (1 - \rho^{t-1}_{e,b}) \cdot \psi_{e,b} \tag{13}$$

Let $\mathcal{B}^e_{t-1}$ be the set of services assigned to server e at timestamp $t-1$. We normalize the current supply vector and the aggregated demand vector separately:

$$\alpha^{t-1}_e = \mathrm{Softmax}\left([p^{t-1}_{e,c}, p^{t-1}_{e,s}, p^{t-1}_{e,b}] \right) \tag{14}$$

$$\beta^{t-1}_e = \mathrm{Softmax}\left(\left[\sum_{s \in \mathcal{B}^e_{t-1}} \varphi_{s,c}, \sum_{s \in \mathcal{B}^e_{t-1}} \varphi_{s,s}, \sum_{s \in \mathcal{B}^e_{t-1}} \varphi_{s,b} \right] \right) \tag{15}$$

The discrepancy between demand and supply is then injected into the previous load state:

$$\gamma^{t-1}_e = \mathrm{Softmax}\left(\mathrm{Softmax}(\beta^{t-1}_e - \alpha^{t-1}_e) + [\rho^{t-1}_{e,c}, \rho^{t-1}_{e,s}, \rho^{t-1}_{e,b}] \right) \tag{16}$$

To account for task completion and resource release, we multiply the intermediate load by a nonlinear release factor $\eta = \tanh(x)$ with $x \sim \mathcal{U}(0,1)$:

$$\rho^t_e = \gamma^{t-1}_e \cdot \eta \tag{17}$$

This produces temporally smooth load series with both accumulation and decay. Sample load records are shown in Table V.

TABLE V
SAMPLE SERVER LOAD RECORDS

timestamp	eid	computing_load	storage_load	bandwidth_load
0	0	14.78%	6.45%	5.67%
140	625	97.63%	11.00%	72.26%
161	1089	6.33%	16.24%	11.15%
213	1270	54.09%	16.01%	50.13%
320	1031	34.21%	7.72%	93.04%

C. Response Time Data Generation

The response time of an invocation is modeled as the sum of six stages: request propagation delay, uplink transmission delay, queueing delay, processing delay, downlink transmission delay, and response propagation delay [34]–[37]. This decomposition makes it possible to link latency to both network geometry and server-side resource dynamics.

1) Request Propagation Delay: The request propagation delay depends on the great-circle distance between the user and the selected server:

$$PG_i^{req} = \frac{\text{Haversine}(\lambda_u^t, \phi_u^t, \lambda_e, \phi_e)}{c} \tag{18}$$

where $c = 3\times10^8$ m/s is the propagation speed of radio waves.

2) Transmission Delay: Bandwidth supply levels are mapped to a discrete bandwidth set $\mathcal{W}_e = \{16, 64, 128, 512, 1024, 2048\}$ Mbps, while service bandwidth demand levels are mapped to packet sizes $\mathcal{W}_s = \{1, 8, 64, 512, 2048, 4096\}$ KB. For a service s assigned to server e at timestamp t, we define

$$B_{e,s}^t = W^e_{\lfloor\psi_{e,b}\rfloor} \cdot (1-\rho_{e,b}^t) \cdot \xi \cdot \frac{\varphi_{s,b}}{\sum_{s'\in\mathcal{B}_t^e}\varphi_{s',b}} \tag{19}$$

$$L_s = (1+\varphi_{s,b} - \lfloor\varphi_{s,b}\rfloor) \cdot W^s_{\lfloor\varphi_{s,b}\rfloor} \tag{20}$$

where $\xi = 128$ is the conversion factor from Mbps to KB/s. The uplink and downlink transmission delays are

$$U_i = \frac{L_s}{B_{e,s}^t} \tag{21}$$

$$D_i = \frac{L_s}{W^e_{\lfloor\psi_{e,b}\rfloor}\cdot(1-\rho_{e,b}^t)\cdot\xi} \tag{22}$$

3) Queueing Delay: Before execution, each invocation waits for computation, storage, and bandwidth resources. We model the three waiting stages with M/M/1 queues [38]. To better separate low-demand and high-demand services, the resource demand levels are first expanded by

$$\tilde{\varphi}_{s,*} = 1.5^{\varphi_{s,*}}, \qquad *\in\{c,s,b\}. \tag{23}$$

For each resource type,

$$\lambda_{e,s,*}^t = \frac{\tilde{\varphi}_{s,*}}{\sum_{s'\in\mathcal{B}_t^e}\tilde{\varphi}_{s',*}} \tag{24}$$

$$\mu_{e,s,*}^t = \lambda_{e,s,*}^t \cdot \big(\psi_{e,*}(1-\rho_{e,*}^t)+1\big) \tag{25}$$

$$\chi_{e,s,*}^t = \frac{\psi_{e,*}(1-\rho_{e,*}^t)}{\tilde{\varphi}_{s,*}} \tag{26}$$

$$Q_{i,*} = e^{-\chi_{e,s,*}^t} \cdot \frac{\lambda_{e,s,*}^t}{\mu_{e,s,*}^t(\mu_{e,s,*}^t - \lambda_{e,s,*}^t)} \tag{27}$$

The total queueing delay is $Q_i = Q_{i,c} + Q_{i,s} + Q_{i,b}$.

4) Processing Delay: After the required resources become available, service execution is governed mainly by computation and storage:

$$P_i = \frac{\tilde{\varphi}_{s,c}}{(1-\rho_{e,c}^t)\psi_{e,c}} \cdot (1+\rho_{e,c}^t) + \frac{\tilde{\varphi}_{s,s}}{(1-\rho_{e,s}^t)\psi_{e,s}} \cdot (1+\rho_{e,s}^t) \tag{28}$$

This form penalizes both high demand and high server utilization.

5) Delay Regularization: The transmission, queueing, and processing terms are generated by different abstractions and therefore lie on different scales. To combine them without allowing extreme values to dominate, CHESTNUT first compresses them with logarithmic transforms and then applies min-max normalization followed by a $\tanh$ nonlinearity:

$$I_i = \log(1+U_i) + \sum_{*\in\{c,s,b\}} \log(1+Q_{i,*}) + \log(1+P_i+D_i) \tag{29}$$

$$\tag{30}$$

Here SD_i is the server-side delay component expressed in milliseconds.

6) Response Propagation Delay: Because the user may have moved while the request is being processed, the return packet is assumed to travel to an estimated future location rather than the original one. Let v_u^t be the user speed in m/s and θ_u^t the travel direction. The estimated travel distance before the response is delivered is

$$d = v_u^t \cdot (PG_i^{req} + SD_i). \tag{31}$$

The predicted receiving position is

$$\hat{\phi}_u^t = \phi_u^t + \frac{d\cdot\cos(\text{rad}(\theta_u^t))}{111320} \tag{32}$$

$$\hat{\lambda}_u^t = \lambda_u^t + \frac{d\cdot\sin(\text{rad}(\theta_u^t))}{111320\cdot\cos(\text{rad}(\phi_u^t))} \tag{33}$$

and the response propagation delay is

$$PG_i^{rep} = \frac{\text{Haversine}(\lambda_e, \phi_e, \hat{\lambda}_u^t, \hat{\phi}_u^t)}{c} \tag{34}$$

Finally, the total response time is

$$RT_i = \kappa\cdot PG_i^{req} + SD_i + \kappa\cdot PG_i^{rep} \tag{35}$$

where $\kappa = 10^4$ is a scale factor used to bring propagation delays to the same observable range as the server-side delay.

D. Network Jitter Data Generation

Network jitter captures delay fluctuation rather than absolute delay [39]. In mobile edge systems, it is influenced by user mobility, bandwidth pressure, and the spatial relation between users and servers [40], [41]. CHESTNUT combines five factors to synthesize jitter.

1) Bandwidth Load Trend: The first factor is the recent trend of server bandwidth utilization:

$$\varsigma_i^t = \frac{1}{|T_e^t|} \sum_{\tau \in T_e^t} \left(\rho_{e,b}^{\tau} - \rho_{e,b}^{\tau-1} \right) \tag{36}$$

where T_e^t contains up to k historical timestamps before t. A positive trend indicates rising bandwidth pressure and hence a less stable transmission environment.

2) User-Server Distance Ratio: The second factor measures how close the user is to the boundary of the serving region:

$$\varsigma_i^d = \frac{\text{Haversine}(\lambda_u^t, \phi_u^t, \lambda_e, \phi_e)}{R_e}. \tag{37}$$

Larger values imply a weaker and potentially less stable connection.

3) Average Directional Change: The third factor quantifies short-term changes in user movement direction:

$$\varsigma_i^c = \frac{1}{|T_u^t|} \sum_{\tau \in T_u^t} |\theta_u^{\tau} - \theta_u^{\tau-1}| \tag{38}$$

where T_u^t contains up to k historical records of user u.

4) Bandwidth Demand Ratio: The fourth factor characterizes the mismatch between service demand and currently available bandwidth:

$$\varsigma_i^r = \frac{\varphi_{s,b}}{\psi_{e,b}(1 - \rho_{e,b}^t)} \tag{39}$$

Large values indicate that the request consumes a large fraction of the server's remaining bandwidth.

5) Jitter Fusion: The fifth factor is the user's instantaneous speed v_u^t. After min-max normalization, the five terms are fused as

$$\varsigma_i = e^{1+\hat{\varsigma}_i^t} \cdot \left(\hat{\varsigma}_i^d + \hat{\varsigma}_i^c + \hat{\varsigma}_i^r + \hat{v}_i \right) \tag{40}$$

$$J_i = \left(\tanh\left(M_{[-2,\ 2]}(\varsigma_i) \right) + 1 \right) \Theta_{NJ} \tag{41}$$

where J_i is the final jitter value in milliseconds.

E. Perturbations

To avoid making QoS values overly deterministic, CHESTNUT injects two additional perturbations. The first is an edge-context perturbation generated from the triplet (u_i, e_i, s_i) by a lightweight feedforward model:

$$\delta_{edge}^{u_i,e_i,s_i} = \text{Model}([u_i, e_i, s_i]) \tag{42}$$

The second is a time perturbation:

$$\delta_{time}^{t_i} = 0.1 \left(\sin\left(\frac{t_i}{2} \right) + 1 \right) \tag{43}$$

Both perturbations are scaled into $[0, 0.2]$, so the final QoS values fluctuate within a moderate but meaningful range:

$$Q_{rt}^i = RT_i \cdot \left(1 + \delta_{edge}^{u_i,e_i,s_i} + \delta_{time}^{t_i} \right) \tag{44}$$

$$Q_{nj}^i = J_i \cdot \left(1 + \delta_{edge}^{u_i,e_i,s_i} + \delta_{time}^{t_i} \right) \tag{45}$$

Table VI lists several sample invocation records.

IV. Results and Analysis

This section reports the main statistics of CHESTNUT and analyzes whether the generated dataset reflects the expected properties of a mobile edge environment.

As summarized in Table VII, CHESTNUT contains full four-dimensional interactions among users, servers, services, and timestamps. Compared with datasets designed for traditional web-service QoS prediction, it also exposes the spatial and dynamic server-side context required by mobile edge research. Table VIII highlights the difference from WS-DREAM.

A. User Mobility and Coverage Characteristics

User continuity and server coverage. After screening, all selected users exhibit fully continuous active timestamps under the aligned system clock, which means the final user trajectories are suitable for temporal QoS modeling rather than sparse event matching. Figure 6 reports the distribution of the average number of users covered by a server.

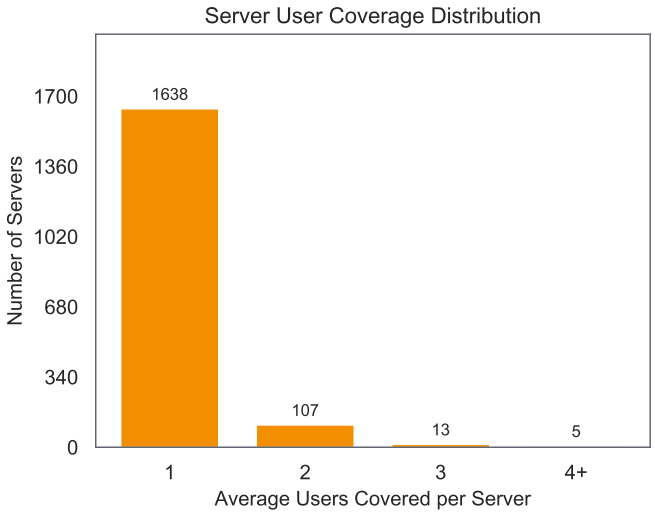

Fig. 6. Distribution of the average number of users covered by each server.

Most servers cover between one and three users on average, and only five servers exceed four covered users. This light but nontrivial overlap matches the spatially scattered behavior expected in urban edge scenarios.

Temporal similarity within request groups. The request synthesis module is intended to generate users with similar global trends inside the same group while preserving local variation at the individual level. Figure 7 illustrates one such pair of users.

TABLE VI
SAMPLE INVOCATION RECORDS

uid	eid	sid	timestamp	rt	nj
0	606	27	0	0.162801	8.963441
54	732	29	0	0.173344	12.486833
8	8	108	3	0.097180	8.400173
54	210	16	77	0.357487	9.464810
77	1689	79	111	0.175501	17.383079

TABLE VII
OVERALL STATISTICS OF CHESTNUT

Item	Value
Users	1,000
Servers	1,763
Services	1,000
Timestamps	720
Invocations	33,551,574

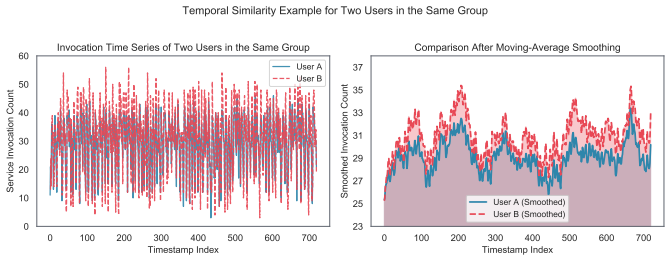

Fig. 7. Example of temporal similarity between two users from the same request-pattern group.

Their raw invocation counts fluctuate differently at some timestamps, yet the smoothed curves remain highly consistent in terms of major rises and falls. This suggests that the grouped Gaussian-process construction captures shared behavioral rhythm without collapsing all users into identical traces.

B. QoS-Related Statistics

Service diversity. Figure 8 shows the demand-level distributions of the 1,000 synthetic services over computation, storage, and bandwidth.

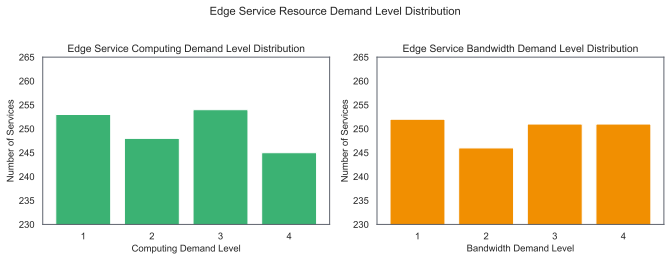

Fig. 8. Demand-level distributions of synthetic services.

The three histograms are close to uniform, with roughly 250 services per level. This balance prevents the dataset from being dominated by either lightweight or heavyweight service types and provides a wide range of operating conditions for QoS prediction.

Temporal server load dynamics. Figure 9 presents the time series of a representative server.

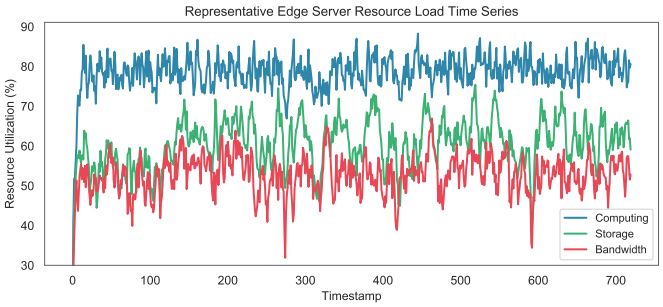

Fig. 9. Example of temporal load evolution for a representative edge server.

The mean utilization of computation, storage, and bandwidth is approximately 78.6%, 60.5%, and 52.8%, respectively. More importantly, the curves evolve smoothly instead of repeatedly jumping between idle and saturated states. This indicates that the recursive load update with nonlinear release successfully preserves temporal dependence, which is crucial for forecasting tasks that rely on recent server history.

Response time distribution. Figure 10 reports both the empirical density and the cumulative distribution of response time.

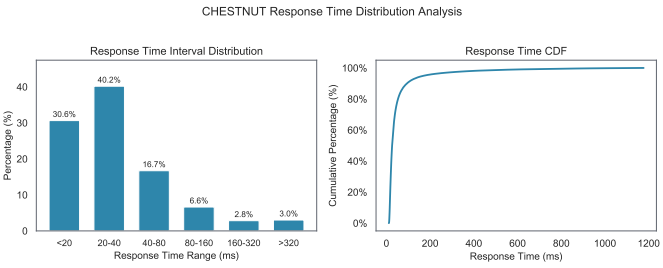

Fig. 10. Probability density and cumulative distribution of response time in CHESTNUT.

About 31.2% of invocations are below 20 ms, and another 40.0% lie between 20 ms and 40 ms, so 71.2% of the records fall into a low-latency regime. The mean response time is about 60.9 ms, whereas the median is 25.8 ms, which indicates a clear right-skewed distribution with a moderate long tail. This behavior is desirable because it covers both common low-latency cases and rarer congested situations.

Network jitter distribution. Figure 11 shows the distribution of network jitter.

TABLE VIII
COMPARISON BETWEEN WS-DREAM AND CHESTNUT

Property	WS-DREAM	CHESTNUT
User ID and service ID	✓	✓
Server ID		✓
Timestamp index	✓	✓
User-service QoS records	✓	✓
User-server-service QoS records		✓
User trajectory sequence		✓
Server location metadata	✓	✓
User-server coverage relation		✓
Server resource supply		✓
Dynamic server load		✓
Service demand attributes		✓
Response time	✓	✓
Throughput	✓	
Network jitter		✓

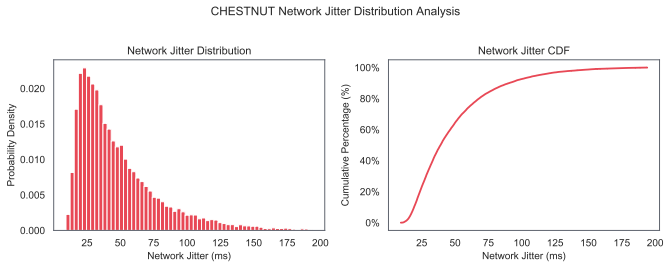

Fig. 11. Probability density and cumulative distribution of network jitter in CHESTNUT.

Around 51.7% of invocation records are below 40 ms, and 34.6% lie between 40 ms and 80 ms. The mean and median are 48.7 ms and 39.0 ms, respectively. Compared with response time, jitter values are more concentrated in the middle range, which is consistent with the fact that most services experience relatively stable transmission while a smaller fraction suffers from stronger mobility- or bandwidth-driven fluctuation.

REFERENCES

[1] K. Kritikos and D. Plexousakis, "Requirements for qos-based web service description and discovery," *IEEE Transactions on Services Computing*, vol. 2, no. 4, pp. 320–337, 2009.

[2] Y. Syu, J.-Y. Kuo, and Y.-Y. Fanjiang, "Time series forecasting for dynamic quality of web services: An empirical study," *Journal of Systems and Software*, vol. 134, pp. 279–303, 2017.

[3] M. Karakus and A. Durresi, "Quality of service (qos) in software defined networking (sdn): A survey," *Journal of Network and Computer Applications*, vol. 80, pp. 200–218, 2017.

[4] L. Cristobo, E. Ibarrola, I. Casado-O'Mara, and L. Zabala, "Global quality of service (qox) management for wireless networks," *Electronics*, vol. 13, no. 16, 2024. [Online]. Available: https://www.mdpi.com/2079-9292/13/16/3113

[5] H. Xue, B. Huang, M. Qin, H. Zhou, and H. Yang, "Edge computing for internet of things: A survey," in *2020 International Conferences on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData) and IEEE Congress on Cybermatics (Cybermatics)*. IEEE, 2020, pp. 755–760.

[6] M. B. Karimi, A. Isazadeh, and A. M. Rahmani, "Qos-aware service composition in cloud computing using data mining techniques and genetic algorithm," *The Journal of Supercomputing*, vol. 73, pp. 1387–1415, 2017.

[7] D. A. Menascé, "Qos issues in web services," *IEEE Internet Comput.*, vol. 6, pp. 72–75, 2002. [Online]. Available: https://api.semanticscholar.org/CorpusID:42937779

[8] K. Shade, A. O. Akinde, O. Ronke, and O. O. Samuel, "Quality of service (qos) issues in web services," 2012. [Online]. Available: https://api.semanticscholar.org/CorpusID:167434500

[9] J. Wu, L. Chen, Y. Feng, Z. Zheng, M. Zhou, and Z. Wu, "Predicting quality of service for selection by neighborhood-based collaborative filtering," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 43, pp. 428–439, 2013. [Online]. Available: https://api.semanticscholar.org/CorpusID:15261484

[10] C. Wu, W. Qiu, X. Wang, Z. Zheng, and X. Yang, "Time-aware and sparsity-tolerant qos prediction based on collaborative filtering," *2016 IEEE International Conference on Web Services (ICWS)*, pp. 637–640, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:837796

[11] J. Li, J. Wang, Q. Sun, and A. Zhou, "Temporal influences-aware collaborative filtering for qos-based service recommendation," *2017 IEEE International Conference on Services Computing (SCC)*, pp. 471–474, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:25892490

[12] Z. Zheng, L. Xiaoli, M. Tang, F. Xie, and M. R. Lyu, "Web service qos prediction via collaborative filtering: A survey," *IEEE Transactions on Services Computing*, vol. 15, pp. 2455–2472, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:219501848

[13] M. Tang, T. Zhang, J. Liu, and J. Chen, "Cloud service qos prediction via exploiting collaborative filtering and location-based data smoothing," *Concurrency and Computation: Practice and Experience*, vol. 27, pp. 5826 – 5839, 2015. [Online]. Available: https://api.semanticscholar.org/CorpusID:24993426

[14] K. Su, L. Ma, B. Xiao, and H. Zhang, "Web service qos prediction by neighbor information combined non-negative matrix factorization," *J. Intell. Fuzzy Syst.*, vol. 30, pp. 3593–3604, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:29886978

[15] Q. Zhang, C. Ding, and C.-H. Chi, "Collaborative filtering based service ranking using invocation histories," *2011 IEEE International Conference on Web Services*, pp. 195–202, 2011. [Online]. Available: https://api.semanticscholar.org/CorpusID:15071119

[16] G. Zou, Y. Huang, S. Hu, Y. Gan, B. Zhang, and Y. Chen, "Trcf: Temporal reinforced collaborative filtering for time-aware qos prediction," *IEEE Transactions on Services Computing*, vol. 17, pp. 1847–1860, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:265124546

[17] H. Wu, K. Yue, B. Li, B. Zhang, and C.-H. Hsu, "Collaborative qos prediction with context-sensitive matrix factorization," *Future Gener. Comput. Syst.*, vol. 82, pp. 669–678, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:3637653

[18] J. Zhou, D. Ding, Z. Wu, and Y. Xiu, "Spatial context-aware time-series forecasting for qos prediction," *IEEE Transactions on Network and Service Management*, vol. 20, pp. 918–931, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:257300058

[19] P. Zhang, W. Huang, Y. Chen, and M. Zhou, "Predicting quality of

services based on a two-stream deep learning model with user and service graphs," *IEEE Transactions on Services Computing*, vol. 16, pp. 4060–4072, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:261149398
[20] Y. Jin, K. Wang, Y. Zhang, and Y. Yan, "Neighborhood-aware web service quality prediction using deep learning," *EURASIP Journal on Wireless Communications and Networking*, vol. 2019, pp. 1–10, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:201838513
[21] C. Awanyo and N. Guermouche, "Deep neural network-based approach for iot service qos prediction," in *WISE*, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:264441803
[22] A. Graves and A. Graves, "Long short-term memory," *Supervised sequence labelling with recurrent neural networks*, pp. 37–45, 2012.
[23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
[24] G. Zou, S. Wu, S. Hu, C. Cao, Y. Gan, B. Zhang, and Y. Chen, "Ncrl: Neighborhood-based collaborative residual learning for adaptive qos prediction," *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 2030–2043, 2022.
[25] N. Koursioumpas, L. Magoula, I. Stavrakakis, N. Alonistioti, M. A. Gutierrez-Estevez, and R. Khalili, "Distinqt: A distributed privacy aware learning framework for qos prediction for future mobile and wireless networks," *ArXiv*, vol. abs/2401.10158, 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID:267035178
[26] G. Zou, S. Lin, S. Hu, S. Duan, Y. Gan, B. Zhang, and Y. Chen, "Fhc-dqp: Federated hierarchical clustering for distributed qos prediction," *IEEE Transactions on Services Computing*, vol. 16, pp. 4073–4086, 2023. [Online]. Available: https://api.semanticscholar.org/CorpusID:261310990
[27] G. Zou, W. Yu, S. Hu, Y. Gan, B. Zhang, and Y. Chen, "Frln: Federated residual ladder network for data-protected qos prediction," *IEEE Transactions on Services Computing*, 2024.
[28] Z. Zheng, Y. Zhang, and M. R. Lyu, "Investigating qos of real-world web services," *IEEE transactions on services computing*, vol. 7, no. 1, pp. 32–39, 2012.
[29] D. Soldani, K. Pentikousis, R. Tafazolli, and D. Franceschini, "5g networks: End-to-end architecture and infrastructure [guest editorial]," *IEEE Commun. Mag.*, vol. 52, pp. 62–64, 2014. [Online]. Available: https://api.semanticscholar.org/CorpusID:33738654
[30] Y. Li, A. Zhou, X. Ma, and S. Wang, "Profit-aware edge server placement," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 55–67, 2021.
[31] Y. Guo, S. Wang, A. Zhou, J. Xu, J. Yuan, and C.-H. Hsu, "User allocation-aware edge cloud placement in mobile edge computing," *Software: Practice and Experience*, vol. 50, no. 5, pp. 489–502, 2020.
[32] S. Wang, Y. Guo, N. Zhang, P. Yang, A. Zhou, and X. Shen, "Delay-aware microservice coordination in mobile edge computing: A reinforcement learning approach," *IEEE Transactions on Mobile Computing*, vol. 20, no. 3, pp. 939–951, 2019.
[33] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. MIT Press, 2006.
[34] X. Chen, W. Wang, and J. Nie, "Analysis of web response time in asymmetrical wireless network," *2008 11th IEEE Singapore International Conference on Communication Systems*, pp. 1427–1430, 2008. [Online]. Available: https://api.semanticscholar.org/CorpusID:43052448
[35] L. Ruan, M. P. I. Dias, Y. Feng, and E. Wong, "Round-trip delay modeling for smart body area networks," *IEEE Communications Letters*, vol. 21, pp. 2528–2531, 2017. [Online]. Available: https://api.semanticscholar.org/CorpusID:4923183
[36] M. Ulvan and R. Bestak, "Delay performance of session establishment signaling in ip multimedia subsystem," *2009 16th International Conference on Systems, Signals and Image Processing*, pp. 1–5, 2009. [Online]. Available: https://api.semanticscholar.org/CorpusID:18626127
[37] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 7th ed. Pearson, 2017.
[38] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley, 1975.
[39] C. Demichelis and P. Chimento, "IP packet delay variation metric for IP performance metrics (IPPM)," RFC Editor, RFC 3393, 2002. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3393
[40] Y. Chen, D. Wang, N. Wu, and Z. Xiang, "Mobility-aware edge server placement for mobile edge computing," *Computer Communications*, vol. 208, pp. 136–146, 2023.
[41] H. Jin, P. Zhang, H. Dong, A. Bouguettaya, and A. Y. Zomaya, "Swift and accurate mobility-aware QoS forecasting for mobile edge environments," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 4340–4353, 2024.

Guobing Zou is a full professor and dean of the Department of Computer Science and Technology, Shanghai University, China. He received his PhD degree in Computer Science from Tongji University, Shanghai, China, 2012. He has worked as a visiting scholar in the Department of Computer Science and Engineering at Washington University in St. Louis from 2009 to 2011, USA. His current research interests mainly focus on services computing, edge computing, data mining and intelligent algorithms, recommender systems. He has published more than 100 papers on premier international journals and conferences, including IEEE Transactions on Services Computing, IEEE Transactions on Network and Service Management, IEEE ICWS, ICSOC, IEEE SCC, AAAI, IJWSR, IJWGS, Information Sciences, Expert Systems with Applications, Knowledge-Based Systems, Applied Intelligence, etc. He served as organization and publicity chair of the International Conference on Service Science, vice chair of IEEE International Conference on Big Data (IEEE BigData 2021), chair of "Service Computing Top Conference Top Journal Forum" of China Digital Service Conference (2021-2023), and guest editor of International Journal of Services Technology and Management.

Fei Zhao received the bachelor's degree in software engineering from Fujian Normal University, China, 2022. He is currently working toward the master's degree with the School of Computer Engineering and Science, Shanghai University, China. His research interests include QoS prediction in mobile edge environment, deep learning, graph neural network. His current work involves the application of spatio-temporal graph neural networks to enhance the accuracy of QoS predictions in dynamic mobile edge scenarios.

Shengxiang Hu received the bachelor degree in 2018 and the master degree in 2021 both in computer science and Technology from Shanghai University, respectively. He is currently working toward the PhD degree in the School of Computer Engineering and Science, Shanghai University, China. His research interests include QoS prediction, graph neural network and natural language processing. He has published more than five papers on IEEE Transactions on Services Computing, IEEE Knowledge-Based Systems, International Conference on Web Services (IEEE ICWS), International Conference on Service-Oriented Computing (ICSOC), International Conference on Parallel Problem Solving from Nature (PPSN).